Reverse Recipe

Aplicación web que analiza la imagen de un plato, identifica ingredientes y genera recetas relacionadas con ayuda de inteligencia artificial.

Cómo funciona

1. El usuario carga una imagen desde el frontend Angular.
2. El frontend la envía al backend Flask mediante `POST /api/scan`.
3. El backend usa AWS Rekognition para detectar etiquetas y conserva las que pertenecen a la categoría de alimentos. Devuelve cada ingrediente con su nivel de confianza.
4. El usuario solicita recetas. Angular envía los ingredientes detectados a `POST /api/recipes`.
5. Flask construye una instrucción en español y llama a Gemini. La respuesta se devuelve como texto y se muestra debajo del formulario.

El frontend se conecta al backend en `http://localhost:5000/api`. La generación espera a que Gemini responda; el backend registra en consola el inicio y la duración de cada solicitud.

Tecnologías

- **Frontend:** Angular, TypeScript y Tailwind CSS.
- **Backend:** Python y Flask.
- **Detección de ingredientes:** AWS Rekognition.
- **Generación de recetas:** Google Gemini mediante `google-genai`.

Requisitos

- Python 3.10 o posterior.
- Node.js y npm compatibles con la versión de Angular del proyecto.
- Credenciales de AWS con acceso a Rekognition.
- Una clave de API de Google Gemini.

Configuración y ejecución

Backend

Desde la raíz del proyecto:

```
cd backend
python3 -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt
cp .env.example .env
```

Edita `backend/.env` y configura, como mínimo:

```
ACCESS_KEY_ID=tu_clave_de_acceso_aws
ACCESS_SECRET_KEY=tu_clave_secreta_aws
REGION=tu_region_aws
GEMINI_API_KEY=tu_clave_de_google_gemini
```

`GEMINI_MODEL` es opcional; si no se define, se utiliza `gemini-3.8-flash`. No publiques ni incluyas las claves en el control de versiones.

Con el entorno virtual activado, inicia Flask:

```
flask --app app run
```

El backend queda disponible en `http://localhost:5000`.

Frontend

En otra terminal, desde la raíz del proyecto:

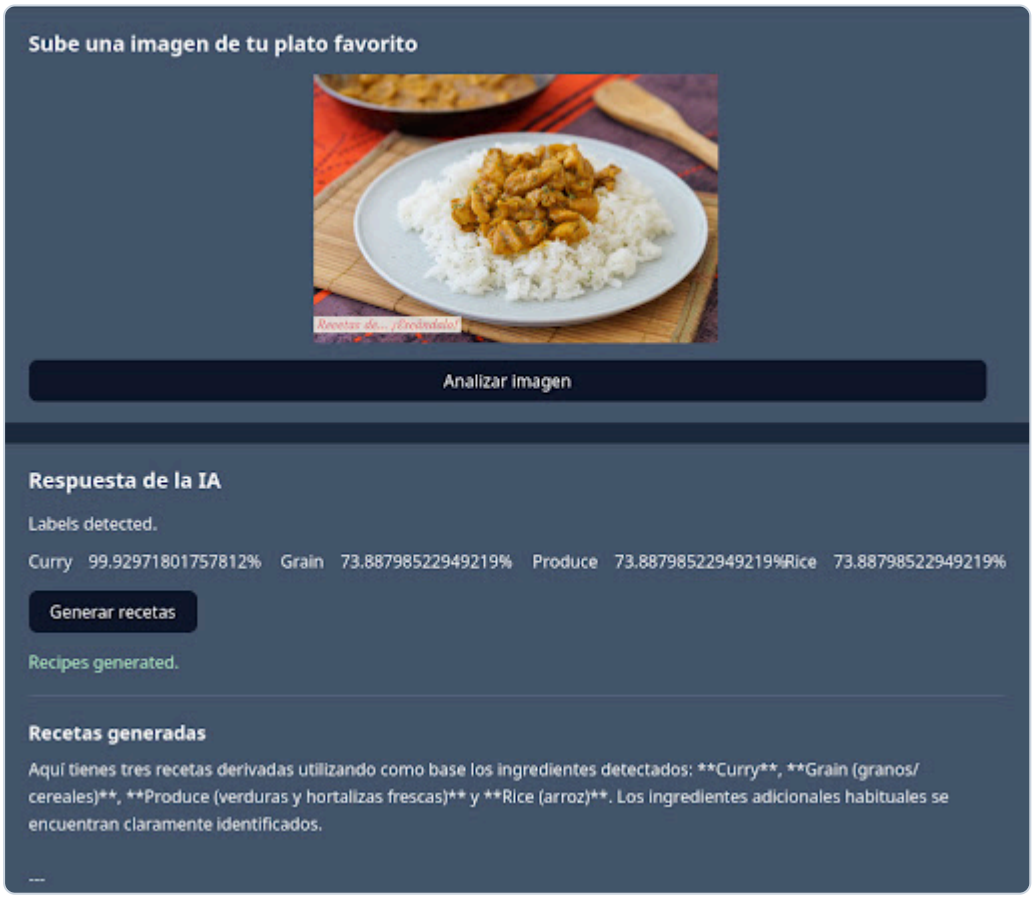
```
cd frontend
npm install
npm start
```

Abre `http://localhost:4200` en el navegador.

API

- `GET /api/`: comprueba que el servidor está activo.
- `POST /api/scan`: recibe una imagen como `multipart/form-data` en el campo `image` y devuelve los ingredientes detectados.
- `POST /api/recipes`: recibe un objeto JSON como `{"text":"tomate, arroz"}` y devuelve el mensaje y el texto de las recetas generadas.

Ejemplo de salida del programa



> Es posible que algunas veces, en caso de que no tengas ningún plan de IA con Gemini tarde bastante ya que tira de servicios gratuitos que van por orden de petición. Incluso habrá veces que devolverá un error por tardar demasiado o que directamente no lo haga.